

ProLink

Project guide: workflow, Computer Networks as part of the project, and the SRS structure. Team of 4, about 3 months. Stack: Next.js on Vercel, Supabase, C# (DSA), Python (Data Science).

How to use this document. Part A explains what the project is and how it works. Part B adds Computer Networks (CN) as a real part of the project: which features to add and what to do for each. Part C is the team plan and schedule. Part D describes the structure of the SRS (headings, diagrams, tables) based on the teacher's sample. Part D gives structure only; the content of the SRS is for the team to write.

Part A. The project: what it is and how it works

A1. Idea in one paragraph

ProLink is a web platform where a customer posts a job (leaking pipe, wiring fault, painting) with category, area, budget and urgency. The system matches and ranks relevant professionals, who send offers. The customer compares offers, accepts one, and both coordinate by in-app chat until the job is done, then the customer leaves a review. Professionals can also publish service listings that customers can browse and request. An admin manages users, categories, reports and sees analytics.

Actor	Does what
Customer	Posts job, compares offers, accepts one, chats, tracks status, reviews
Professional	Builds profile, gets matched to jobs, sends offers, chats, updates status, publishes services
Admin	Verifies professionals, manages categories, handles reports, views dashboards and network health
System (automatic)	Top-K matching, notifications, job expiry, category suggestion, health monitoring

A2. Feature list (MoSCoW)

Priority	Features
MUST	Register/login with roles professional profile post job job lifecycle state machine browse and search open jobs top-K matching and notifications one offer per professional per job compare and accept an offer in-app chat reviews after completion only admin users/categories/dashboard seed data and demo accounts
MUST (CN)	Email notifications through our own SMTP client admin Network Health page HTTPS/security headers and rate limiting packet analysis report Packet Tracer deployment network design (details in Part B)
SHOULD	Verification badge service listings with direct requests category suggestion from job text (classifier) nearest professional by travel time (graph) autocomplete professional analytics report/block user offer withdrawal job auto-expiry TCP live status relay (CN)
COULD	Price hint Roman Urdu text support favorites availability calendar photo attachments in chat load/performance study (CN)
WON'T	Online payments or escrow native mobile app video calls automated identity verification multi-city AI chatbot

- **Explicit job states:** Draft, Open, Offers received, Assigned, In progress, Completed, plus Cancelled, Expired, Disputed.
- **Reviews only after completion**, shown as a Bayesian average so one 5-star review does not outrank 50 reviews averaging 4.7.
- **Contact privacy:** phone number visible only after an offer is accepted. **Urgency flag:** urgent jobs matched first. **Audit table** (job_events) for every state change.
- **Small scope:** 6 to 8 trades, one city, 30 to 60 named areas.

A3. How it works: the three journeys

Journey	Steps
Customer	1 Register and complete profile 2 Post job (title, description, area, budget, urgency); system suggests a category 3 System matches top-K professionals and notifies them 4 Offers arrive (price, inspection fee, ETA); email and in-app notification 5 Customer sorts offers and accepts one; others are rejected 6 Chat to coordinate; phone number revealed 7 Professional marks work done; customer confirms 8 Customer leaves a review
Professional	1 Register, fill skills, areas, availability; optionally upload proof for the verified badge 2 Receive match notifications or browse/search open jobs 3 Send one offer per job 4 If accepted: chat, do the work, mark in progress and done 5 Receive review; see own analytics 6 Optionally publish a service listing that customers can request directly
Admin	1 Verify professionals 2 Manage categories and users 3 Review reports and act 4 Watch platform dashboard (jobs by status, category, area) and the Network Health page

A4. Architecture and who owns what

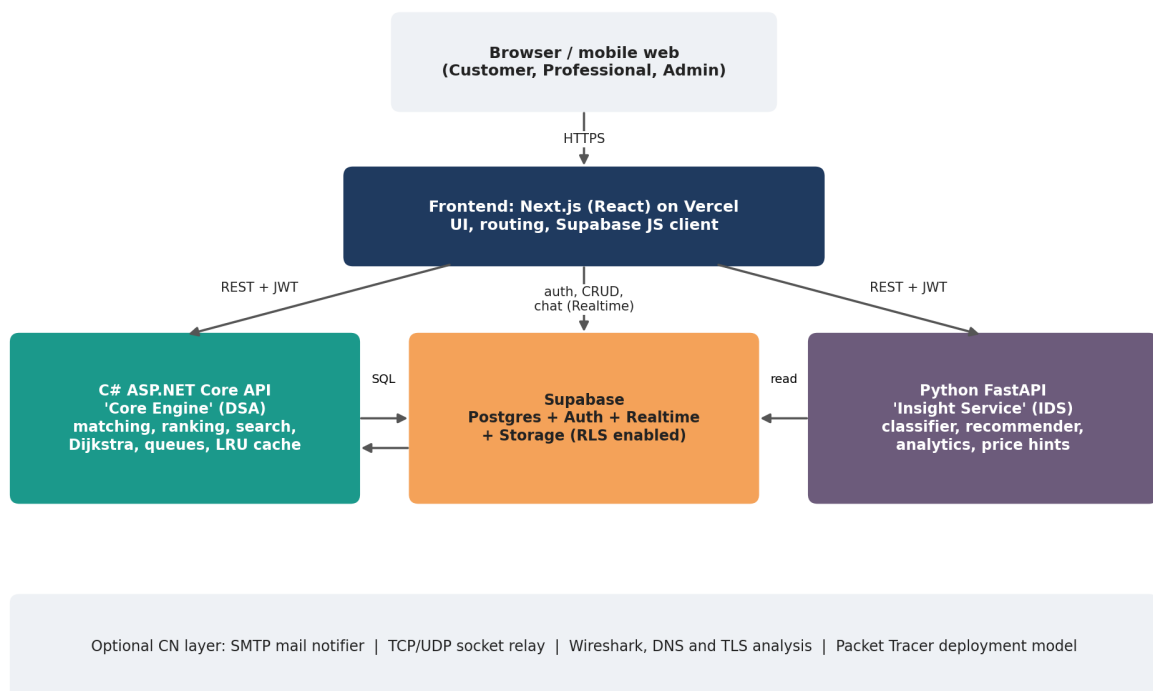


Figure 1. Frontend talks to Supabase directly for simple data and to two small backends for the intelligent parts. The optional CN layer (SMTP, TCP/UDP, Wireshark, Packet Tracer) sits beside it.

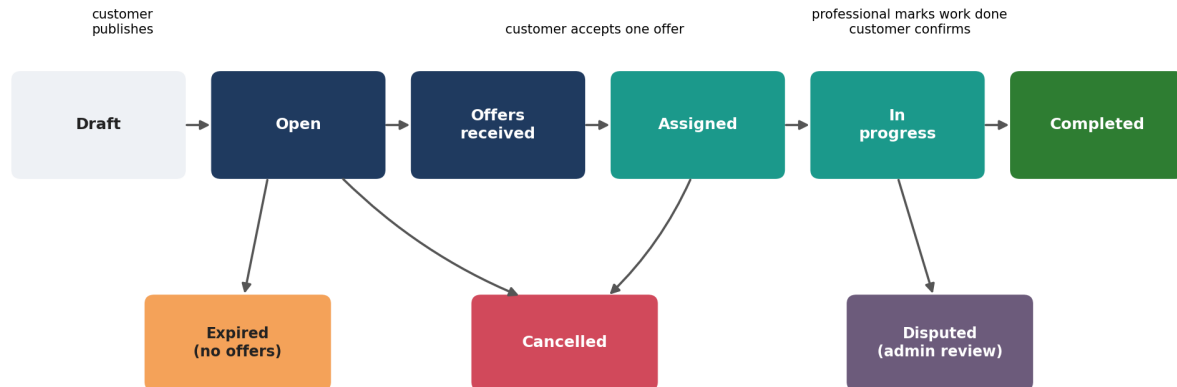
Concern	Where it lives
Signup, login, sessions	Supabase Auth (do not build authentication yourself)
Simple reads/writes (profiles, listings, notifications)	Next.js to Supabase directly, protected by Row Level Security (RLS)
Chat	Supabase Realtime + messages table (RLS: only the two participants)
Job writes, offers, state transitions	C# API (business rules in one place, unit tested)
Search, matching, ranking, nearest professional, cache, notification queue	C# API with in-memory DSA engine loaded from the database
Category suggestion, text similarity, scoring, analytics	Python FastAPI (pandas, NumPy, scikit-learn)
Email, health monitor, TCP relay (CN)	C# network module (separate project in the same solution)
Files (photos, verification documents)	Supabase Storage, private bucket for documents

Request flow when a customer posts a job:

1. Frontend sends POST /jobs with the Supabase JWT to the C# API, which validates it and may ask Python /classify for a category suggestion (the customer confirms).
2. Job is stored as Open and an event is written to job_events.

3. DSA engine finds candidates (inverted index by category and area), computes travel time with Dijkstra, optionally asks Python /rank for text-similarity scores.
4. A bounded min-heap keeps the top K; notifications go through a priority queue (urgent first), are written as rows and pushed by Supabase Realtime; the CN mail module sends the email.
5. If Python is slow or down, the C# engine falls back to default weights and the job still posts.

A5. Job lifecycle and data model



Reviews are allowed only after Completed. Rule enforced in the C# API and by a database constraint.

Figure 2. Job lifecycle. Transitions are enforced in the C# API and recorded in `job_events`.

Table	Key columns
profiles / professional_profiles	id, role, full_name, phone, area_id bio, years_experience, availability, verified, avg_rating_bayes, response_rate
categories / professional_skills	id, name, parent_id profile_id, category_id, skill_tags
areas / area_edges	id, name, lat, lng from_area, to_area, minutes (graph for Dijkstra)
jobs	id, customer_id, title, description, category_id, area_id, budget_min, budget_max, urgency, status, created_at, expires_at
offers	id, job_id, professional_id, price, inspection_fee, eta_minutes, message, status (unique job + professional)
service_listings	id, professional_id, category_id, title, price_min, price_max, area_ids
conversations / messages	id, job_id, customer_id, professional_id id, conversation_id, sender_id, body, created_at
reviews	id, job_id (unique), reviewer_id, professional_id, rating, comment (job must be Completed)
notifications / reports / job_events	user_id, type, payload, read_at reporter_id, target, reason, status job_id, from_status, to_status, actor_id, at
health_checks (CN)	id, target, checked_at, status, rtt_ms, dns_ms, tls_ms

A6. DSA plan (C#)

Each structure must serve a real feature, be implemented by you, unit tested, and have its complexity measured. Confirm with your instructor which built-in types are allowed.

Product feature	Structure / algorithm	Complexity	Proof
Autocomplete	Trie	$O(L)$	Tests; latency vs LINQ scan
Keyword search	Inverted index (hash map + set intersection)	About sum of posting sizes	Benchmark at 1k, 10k, 100k records
Top-K professionals	Bounded min-heap of size K	$O(n \log K)$	Correctness vs full sort
Sort offers	Own stable merge sort, multi-key comparator	$O(n \log n)$	Tests vs <code>Array.Sort</code>
Nearest professionals	Weighted graph + Dijkstra with binary heap	$O((V+E) \log V)$	Hand-checked paths
Urgent-first notifications	Priority queue + FIFO queue per user	$O(\log n)$	Simulation: no starvation

Product feature	Structure / algorithm	Complexity	Proof
Hot query cache	LRU cache (hash map + doubly linked list)	O(1)	Hit-rate counters
Rate limiting (also used in CN security)	Sliding-window queue per user/IP	O(1) amortized	Tests with fake clock

- Build a standalone library (ProLink.Engine) with no database dependency; the API calls into it. Use xUnit (70%+ coverage) and a Stopwatch/BenchmarkDotNet harness for timing tables.
- **Honest framing:** for a few thousand rows a Postgres index beats a hand-written trie. Justify structures by learning outcomes, in-memory ranking that combines signals SQL cannot (graph distance, scoring, caching), and measured behavior at larger synthetic scale.

A7. Intro to Data Science plan (Python)

Data source	What	Honesty rule
Real survey	Google Form in hostels/classes: services hired, price paid, urgency, trust concerns; target 150+ responses	Report sample size and bias
Synthetic generator	Python script (Faker + templates) producing jobs, professionals, offers, reviews shaped by the survey	Label it; never report metrics from synthetic data alone
Pilot logs	10 to 15 classmates over about two weeks (weeks 9 to 11)	Use as the held-out evaluation set

Task	Method	Baseline to beat	Metrics
Suggest category from job text	TF-IDF + logistic regression / linear SVM / naive Bayes	Keyword rules; majority class	Accuracy, macro-F1, confusion matrix, 5-fold CV
Rank professionals	Weighted score: TF-IDF cosine, travel time, Bayesian rating, response rate, availability	Rank by rating only; by distance only	Precision@5, NDCG@5 on about 100 labeled pairs
Fair rating	Bayesian average $(v/(v+m)) \times R + (m/(v+m)) \times C$	Raw mean	Rank stability on low-vote professionals
Dashboards	Demand, funnel (posted, offered, accepted, completed), time to completion	n/a	Reproducible queries

Pipeline: clean (pandas) then explore (EDA) then model with a baseline then evaluate then serve through FastAPI then report in a notebook. Watch for data leakage, class imbalance and overfitting to templated synthetic text.

A8. Hosting facts and security rules

Topic	What to know
C# on Vercel	No first-party .NET runtime; ASP.NET Core runs as a Docker image. Do a 1-day spike in week 1 and shortlist a second container host. The in-memory index must reload from the DB on start.
Python on Vercel	FastAPI deploys with zero configuration; keep dependencies lean (bundle size limit).
WebSockets on Vercel	Use Supabase Realtime for chat, not Vercel functions.
Raw TCP/UDP servers	Vercel functions cannot accept raw TCP/UDP connections. The CN socket modules run locally, on a container host or a VM.
Supabase free plan	500 MB DB, 200 concurrent Realtime connections; projects pause after a week of inactivity. Schedule a daily query and wake it before demos.

Hosting limits change; recheck the official Vercel and Supabase pages before committing.

- RLS on every table with at least one test per policy. Service-role key only in server environments, never in frontend, repo or logs.
- The C# API verifies the JWT on every request and takes the user id from the token, not the request body.
- Validate all input on the server; render chat as text, never raw HTML. Documents in a private bucket readable only by admins.

A9. Bad approaches to avoid

Bad approach	Better approach
Three disconnected mini-projects glued by a login page	One domain: C# does matching/ranking, Python does classification/analytics, CN covers network layer, SRS covers all
UI and chat first, DSA and data science last	Thin vertical slices; build DSA library and Python pipeline in parallel from week 2
Microservices for four students	Exactly two backends (C# API, Python service); simple CRUD straight to Supabase with RLS
Calling List.Sort / Dictionary "DSA"	Implement your own structures with tests and complexity analysis
"AI-powered" that is if-else rules; 95% accuracy on synthetic data	Baseline vs trained model with proper metrics; evaluate on real held-out data
Replacing Supabase Realtime with a hand-written socket chat	Keep Realtime for chat; put raw sockets in the separate CN module
Service-role key in frontend, or RLS off	RLS everywhere; secrets in environment variables only
Writing the SRS at the end	Baselines in weeks 2, 6, 10, 12; requirement IDs in issues, commits and tests
Broadcasting every job to every professional	Match top-K and cap notifications per day
Empty marketplace on demo day	Seed 40 to 60 labeled demo professionals and 100+ jobs, plus a 2-week pilot

Part B. Computer Networks as part of ProLink

Your app already uses HTTPS, REST and WebSockets through frameworks, which counts as using networking, not studying it. To earn CN marks the project must contain protocol-level work or analysis you can defend in a viva. This part lists the CN features to add, what to build for each, when, and how they appear in the SRS.

Principle: the CN modules sit beside the product. They add features users and admins can see, but the product must still work if a CN module is switched off. Do not replace Supabase Realtime or Supabase Auth with hand-written network code.

B1. CN features to add

ID / Feature	What the user or admin sees	CN concepts	What to build	Evidence for report
FR-NET-01 Email notifications via own SMTP client	Customer and professional get an email for new offer, acceptance, status change	TCP, application protocol (SMTP), ports 25/587, STARTTLS, base64 AUTH	C# Smtplib class over TcpClient: EHLO, STARTTLS, AUTH, MAIL FROM, RCPT TO, DATA, QUIT. Test first against a local fake SMTP server, then a real provider on port 587	Wireshark capture of the full SMTP session with each command explained
FR-NET-02 Live status relay over TCP (Should)	A live monitor window (console or small client) shows job status events as they happen	Sockets, client-server, framing, concurrency, keep-alive	C# TcpListener broadcaster: length-prefixed messages, many clients handled with async tasks, heartbeat, clean disconnects. Not on the critical path	Test with 10+ clients; packet capture showing framing; notes on reliability
FR-NET-03 Network Health page for admin	Admin sees up/down, response time, DNS time, TLS time for frontend, C# API, Python service, Supabase	ICMP/ping, UDP heartbeat, DNS resolution, TLS handshake, RTT, latency	C# background service: UDP heartbeat between services, Ping, DNS lookup and TLS handshake timing with Stopwatch; results saved to health_checks table; chart on admin page. Daily query also stops the Supabase pause	Charts of RTT/DNS/TLS over a week; explanation of each metric
FR-NET-04 Network security layer	HTTPS only; blocked abusive traffic; clear error on too many requests	TLS, certificates, HSTS, CORS, rate limiting, firewall rules	Security headers (HSTS, CSP, X-Content-Type-Options), CORS allowlist, per-IP and per-user rate limiter (reuses the DSA sliding-window queue), documented firewall rules	Header scan results; test showing request 101 is refused; certificate details

ID / Feature	What the user or admin sees	CN concepts	What to build	Evidence for report
Packet analysis study (deliverable)	Report section	DNS, TCP 3-way handshake, TLS handshake, HTTP, WebSocket upgrade, SMTP	Wireshark captures against the deployed app. Compare HTTP polling vs WebSocket for the same chat: bytes and latency per message	Annotated screenshots of each protocol and a comparison table
Deployment network design (deliverable)	Diagram and report in the SRS	IP addressing, subnetting/VLSM, VLANs, NAT, DHCP, DNS, ACLs, DMZ, static routing/OSPF	Packet Tracer topology: ISP, edge router with NAT, firewall/ACL, DMZ (web, API, mail, DNS servers), internal VLANs (dev, admin, staff), DHCP, addressing table, routing config	Topology screenshot, addressing table, configs, ping/traceroute test results
Performance study (Could)	Report section	Latency, throughput, caching, compression, CDN	Load test (k6) with and without caching/compression headers; measure effect on page load and API response	Before/after graphs

B2. What to do, step by step

- Week 1:** ask the CN instructor in writing: may CN marks come from a component inside a shared project, what are the minimum deliverables (Packet Tracer, socket code, report, viva), are there group-size limits, are specific protocols required. Record the answer in /docs/decisions.
- Week 2:** choose one CN lead (P1 suggested, with P3 reviewing the C# code). Add the FR-NET requirements and the CN constraint to the SRS scope.
- Weeks 3 to 5:** design the Packet Tracer topology and addressing plan. This needs no product code, so it fits early while the product is still being built.
- Weeks 6 to 7:** build Smtplite and the local test server setup; capture the session in Wireshark.
- Weeks 7 to 8:** build the Health Monitor (UDP, ping, DNS, TLS timing) and the admin Network Health page; then the TCP relay if time allows.
- Weeks 8 to 9:** add security headers, CORS allowlist and the rate limiter; run the header and rate-limit tests.
- Weeks 9 to 10:** capture and annotate the protocol traces; finish the Packet Tracer tests; write the CN report. Feature freeze is the end of week 10.

Effort: about 30 to 40 hours in total. It runs in parallel with the core features, so the team average rises from about 9.2 to about 9.9 hours per person per week.

B3. How CN shows up in the SRS

- Chapter 1 scope and constraints: state that CN is part of the project and list the CN modules.
- Chapter 3: a functional requirement group FR-NET (email, live relay, health monitor, security layer); non-functional requirements for security, latency and availability; software and network requirements.
- Chapter 4: sequence diagram for the email notification flow, a deployment diagram, the network topology diagram with addressing table, and the new health_checks table in the ERD.
- Appendix: Wireshark captures, Packet Tracer configs, test results.

B4. Limits and fallback

- Raw TCP/UDP modules cannot run on Vercel; run them locally or on a container host/VM and demo them from there. Product features must never depend on them.
- Port 25 is often blocked; use a local test SMTP server for the protocol demonstration and port 587 or a provider API for real delivery.
- If you fall behind, cut in this order: performance study, TCP relay, UDP/health page. Never cut the packet analysis, the Packet Tracer design or the SMTP client; they are your main CN evidence.

Part C. Team, schedule and risks

C1. Roles

Role	Primary ownership	Also
P1: Product lead + SRS owner + data layer	Requirements, SRS, backlog, Supabase schema, RLS, CI, seed data	CN lead (topology, packet analysis, report)
P2: Frontend lead	Next.js app, UX flows, wireframes, Vercel deployment, chat UI, admin Network Health page UI	Usability testing
P3: C# engine lead	ASP.NET Core API, DSA library, JWT, matching, benchmarks, DSA report	C# deployment; reviews the CN socket code
P4: Data science lead	Survey, generator, EDA, models, FastAPI, IDS report	Python deployment; health-check data and charts

Every module has one owner and one reviewer; nobody is the only person who can deploy anything. Each person also writes the SRS sections of their own module.

C2. Effort budget (about 475 hours)

Module	Hours
SRS and documentation (all versions, diagrams, traceability)	50
Setup, CI, deployment	25
Authentication, roles, profiles	20
Jobs, offers, lifecycle state machine	45
Service listings	20
Chat (Supabase Realtime)	20
Reviews and notifications	25
Admin features and dashboard	25
DSA engine, tests, benchmarks	55
Data science pipeline, models, FastAPI	55
Frontend polish and UX	40
Testing and QA	30
Course reports and demo preparation	30
Computer Networks module (Part B)	35
Total	about 475

C3. 12-week schedule

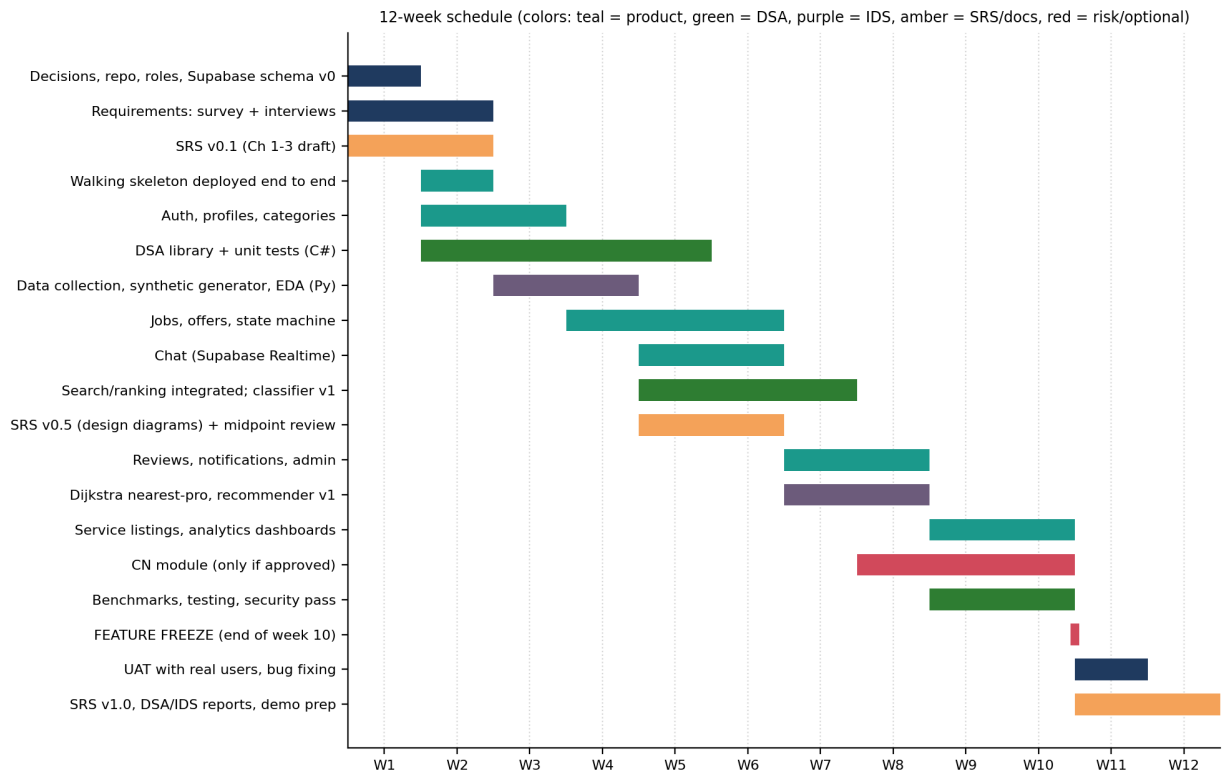


Figure 3. Schedule overview. The red bar is the CN module; the red marker is the feature freeze at the end of week 10.

Wk	Product	DSA (C#)	IDS (Python)	CN	SRS / docs	Gate
1	Decisions, repo, Supabase schema v0, wireframes	Solution skeleton, xUnit	Survey live; data plan	Ask CN instructor	Vision, scope, glossary; interviews	Instructor answers in writing
2	Auth and roles; frontend deployed	API /health and JWT	FastAPI /health	Pick CN lead; FR-NET draft	Use cases, FR/NFR v1: SRS v0.1	Walking skeleton live
3	Profiles, categories, job form	Trie, hash map, queues, heap	Generator, cleaning, EDA	Topology + addressing plan	ERD v1; refine FRs	
4	Job list/detail; browse	Inverted index, merge sort, LRU	Survey results; EDA charts	Packet Tracer build	Screen/interface specs	Sprint demo 2
5	Offers UI; state machine	Matching skeleton, top-K	Classifier baseline vs TF-IDF	Packet Tracer tests	Architecture, DFDs, sequence	
6	Chat via Realtime; notifications v1	Search + matching integrated	Classifier served	SmtpLite + test server	State and activity diagrams: SRS v0.5	Midpoint review + cut-list
7	Reviews; admin basics	Graph + Dijkstra	/rank v1 + Bayesian rating	Health monitor start	Class diagram, API tables, traceability v1	
8	Admin dashboard + Network Health page	Priority queue; benchmarks	Analytics queries	TCP relay; security headers	Test plan skeleton	Sprint demo 4
9	Service listings; start pilot	Benchmarks; complexity report	Model evaluation	Rate limiter; Wireshark captures	Test cases mapped to FRs	
10	UX polish; security pass	Hardening	Final evaluation	CN report	Deployment + network diagrams: SRS v1.0	FEATURE FREEZE

Wk	Product	DSA (C#)	IDS (Python)	CN	SRS / docs	Gate
11	Bug fixing; UAT with 10 to 15 users	Final report	Final report	Fixes	UAT results, change log	UAT done
12	Buffer; two demo rehearsals; backup video	Submission	Submission	Submission	Proofread, consistency, export	Submit

C4. If you run late: cut-list (decide at the week 6 review)

Record every cut in the SRS change log. Order: 1) all Could items; 2) CN performance study, then TCP relay, then UDP health page; 3) service listings and direct requests; 4) nearest-professional graph (fall back to same-area matching); 5) live Python service (use batch scoring); 6) professional analytics dashboard.

C5. Working agreements

- One Git monorepo: /web, /api (Engine, Api, Tests, Network), /insight, /db, /docs. Protected main, pull requests with one reviewer, CI on every PR.
- Definition of Done: reviewed, tests pass, deployed to preview, linked to a requirement ID, SRS updated if behavior changed, no secrets in the diff.
- Contracts first: agree request/response JSON in /docs/api before building a cross-service feature.
- Weekly planning, two short async stand-ups, a demo every two weeks. Mark exam weeks in week 1.

C6. Top risks

Risk	Mitigation
Cross-stack integration slips (JWT, CORS, deployments)	Walking skeleton by week 2; if not live by week 3, pause features
Free-tier pause or cold start during demo	Daily health check, wake the day before, backup video, local docker-compose run
CN work competes with core features	Topology early (no code needed); CN cut-list; one named CN lead
Scope creep	MoSCoW frozen at week 2; changes through the change log
Member unavailable	Every module has an owner and reviewer; runbooks
SRS lags behind the code	Baselines in weeks 2, 6, 10, 12; SRS review is a demo gate

C7. First seven days

1. Day 1: read this together, pick roles and CN lead, set meeting times, mark exam weeks.
2. Day 1 to 2: send the questions below to all four instructors in writing and save replies in /docs/decisions.
3. Day 2: create the monorepo, Supabase project (note the 7-day pause) and Vercel project; protect main; add CI skeletons.
4. Day 2 to 4: spike C# hosting and Python deployment; P2 builds a Next.js hello-world with Supabase login.
5. Day 3: publish the survey, book 5 interviews, start SRS vision, scope and glossary.
6. Day 4 to 6: schema v0 and RLS; wireframes for post-job and offers screens.
7. Day 7: week 1 review; record the CN decision in the SRS scope; plan the week 2 walking skeleton.

To	Questions
DSA instructor	May one combined project be shared with IDS? Which structures are required? May built-in collections be used? Is a web API acceptable as the host of the C# code? What must the report contain?
IDS instructor	Is clearly labeled synthetic data acceptable? Required libraries and deliverables? Minimum model types and metrics? Is a deployed API required?
CN instructor	May CN marks come from a component inside a shared project? Minimum deliverables? Group-size limits? Same codebase graded in several courses? Required protocols or tools?
SE instructor	Is the sample's 4-chapter structure mandatory? Required diagram types and tools? Is the implementation graded with the SRS? Required checkpoints and versions?

Part D. SRS structure guide (structure only, no content)

This part tells you which headings, diagrams and tables the SRS should have. It does not write the SRS; the team writes the content. Rule: requirements say WHAT the system must do, design says HOW. Ask the SE instructor if the sample's structure is mandatory; the plan below keeps it and extends it.

D1. The teacher's sample (RFID attendance FYP) versus what ProLink needs

Part of the sample	Keep / change	For ProLink
Front pages: Final Approval, Declaration, Dedication, Acknowledgement, Abstract, Table of Contents, List of Figures, List of Tables, List of Abbreviations	Keep all	Add Version History and Team Roles
1.1 Introduction, 1.2 Background, 1.3 Scope, 1.4 Motivation and challenges, 1.5 Goals and Objective	Keep	Scope must list in-scope, out-of-scope and the CN decision
1.6 What is RF-ID	Replace	Technologies and key concepts used (Supabase, Next.js, C#, FastAPI, WebSocket, SMTP)
2.1 Introduction, 2.2 Literature Review, 2.3 Summary	Keep and tighten	Add a comparison table of similar platforms; avoid long generic text
3.1 Introduction, 3.2 Problem Scenarios	Keep	Scenarios for customer, professional, admin
3.3 Functional Requirements (short paragraphs, no IDs)	Deepen	Requirement IDs, priority, actor, acceptance criteria, grouped by module
3.4 Non-Functional (Performance, Availability, Reliability, Maintainability, Usability, Safety/Authentication)	Keep headings, make measurable	Add Security, Privacy, Compatibility, Capacity
3.5 Hardware Requirements	Replace	Software and network requirements
4.1 Introduction, 4.2 Architectural Design, 4.3 Detailed Design	Keep	Architecture figure plus table of components
4.4 DFD, 4.5 Use Case (Individual), 4.6 Activity, 4.7 State, 4.8 Swim Lane, 4.9 Sequence, 4.10 Class, 4.11 ERD	Keep all	One short explanation per diagram plus the requirement IDs it covers
Not in the sample	Add	Deployment diagram, network topology, data dictionary, API list, wireframes, DSA and data science design, glossary, traceability matrix, test plan, risks, change log, references

D2. Recommended table of contents (copy this)

Chapter	Sections to include ([NEW] = not in the sample)
Front matter	Title page Final Approval Declaration Dedication Acknowledgement Abstract Table of Contents List of Figures List of Tables List of Abbreviations Version History [NEW]
1 Introduction	1.1 Introduction 1.2 Background 1.3 Scope (in / out / deferred) 1.4 Motivation and challenges 1.5 Goals and Objectives 1.6 Technologies and key concepts 1.7 Stakeholders and user classes [NEW] 1.8 Definitions and glossary [NEW] 1.9 Document conventions: ID scheme, MoSCoW [NEW]
2 Literature Review	2.1 Introduction 2.2 Literature review 2.3 Comparison of similar platforms (table) [NEW] 2.4 Gap and what makes ProLink different [NEW] 2.5 Summary
3 Requirements	3.1 Introduction 3.2 Problem scenarios 3.3 Product perspective and context diagram [NEW] 3.4 User classes and permissions [NEW] 3.5 Functional requirements by module: Authentication, Profile, Job, Matching and Search, Offer, Chat, Review, Notification, Admin, Analytics, Network (CN) 3.6 Non-functional requirements 3.7 External interfaces: UI, API, database, communication protocols [NEW] 3.8 Software and network requirements 3.9 Constraints, assumptions, dependencies [NEW]
4 System Design	4.1 Introduction 4.2 Architectural design 4.3 Detailed design 4.4 DFD 4.5 Use case (individual) 4.6 Activity 4.7 State 4.8 Swim lane 4.9 Sequence 4.10 Class 4.11 ERD 4.12 Data dictionary [NEW] 4.13 API design [NEW] 4.14 UI wireframes [NEW] 4.15 DSA design (C#) [NEW] 4.16 Data science design (Python) [NEW] 4.17 Deployment diagram [NEW] 4.18 Network design: topology and addressing (CN) [NEW]
5 Verification [NEW]	5.1 Test strategy 5.2 Test cases 5.3 Traceability matrix 5.4 UAT plan and results

Chapter	Sections to include ([NEW] = not in the sample)
6 Risks and Change [NEW]	6.1 Risk register 6.2 Change log 6.3 Open issues
End matter [NEW]	References (IEEE style) Appendices: survey and interviews, API tables, CN captures and configs, benchmark and model reports

If the instructor insists on exactly 4 chapters, put Verification and Risks into Appendices A and B instead of Chapters 5 and 6.

D3. Diagrams and tables checklist

Diagram / table	Section	What it should show for ProLink	Tool	Draw in week
Context diagram	3.3	ProLink as one box with customer, professional, admin, Supabase, email server	draw.io	2
Use case diagram + specification tables	4.5	Use cases per actor; one spec table per Must use case	draw.io / StarUML	2 to 3
DFD level 0, 1, 2	4.4	Data flow between actors, processes and data stores (post job, match, offer, chat)	draw.io	5
Architecture diagram	4.2	Frontend, C# API, Python service, Supabase, CN module	draw.io	5
Activity diagram	4.6	Post job to hire to complete to review	draw.io / Mermaid	6
State diagram	4.7	Job lifecycle (and offer status)	draw.io / Mermaid	6
Swim lane diagram	4.8	Customer, professional, system, admin lanes for the main flow	draw.io	6
Sequence diagrams	4.9	Post job and match submit and accept offer chat message email notification (SMTP)	Mermaid / draw.io	5 to 7
Class diagram	4.10	C# engine and API classes (jobs, offers, matching, structures)	StarUML / draw.io	7
ERD + data dictionary	4.11 to 4.12	All tables, keys, relationships; column type and constraint table	dbdiagram.io / draw.io	3 and 7
API table	4.13	Method, path, request, response, errors, role per endpoint	Markdown table	7
Wireframes	4.14	Main screens: login, post job, job list, offers, chat, profile, admin	Figma	1 to 4
Deployment diagram	4.17	Where each part runs (Vercel, container host, Supabase, local CN modules)	draw.io	10
Network topology + addressing table	4.18	Packet Tracer topology with VLSM table and routing	Cisco Packet Tracer	4 to 5, final 10
Traceability matrix	5.3	Requirement to use case to design element to code module to test case	Spreadsheet / table	7, updated each sprint
Comparison table	2.3	Similar platforms versus ProLink by feature, trust mechanism, pricing, gaps	Table	3

D4. What every item should contain (fields only)

Item	Fields to include
Functional requirement	ID (FR-MODULE-NN) statement ("The system shall...") priority (M/S/C) actor source (use case, survey, interview) acceptance criteria related use case related test case
Non-functional requirement	ID category measurable target (a number and how it is measured) how it will be tested
Use case specification	ID and name actor goal preconditions trigger main flow alternative flows postconditions related requirements related tests
Test case	ID linked requirement steps input data expected result actual result status
Risk	ID description likelihood impact mitigation owner trigger
Change log entry	Date requirement ID what changed reason impact approved by
Diagram	Figure number and caption short explanation in your own words requirement IDs it realizes

D5. How to write the SRS alongside the build

Weeks	SRS work	Baseline
1 to 2	Stakeholders, vision, scope, glossary, interviews and survey, use case list, first FR and NFR sets	v0.1 (end of week 2)
3 to 4	Refine FRs as screens and APIs appear; ERD v1; interface specs; literature table	Working draft
5 to 6	Architecture, DFDs, sequence, state, activity, swim lane diagrams	v0.5 (week 6): midpoint review
7 to 8	Class diagram, API tables, data dictionary, traceability matrix v1, test plan skeleton	Working draft
9 to 10	Test cases mapped to FR IDs, deployment and network diagrams, risks; make the document match the code at the freeze	v1.0 (end of week 10)
11 to 12	UAT results, change log, numbering and cross-reference check, export to PDF or Word	Final (week 12)

1. Identify stakeholders and user classes; elicit requirements with a survey, 5 or more interviews and a competitor review (keep notes for the appendix).
2. Write one page of scope with an explicit out-of-scope list.
3. Model use cases; derive atomic functional requirements from them; write measurable non-functional requirements.
4. Specify interfaces (screens, endpoints, tables), then draw the design diagrams once requirements are stable; keep diagram sources in Git.
5. Build the traceability matrix and update it every sprint; peer-review with a checklist (unambiguous, testable, atomic, consistent, necessary, feasible); baseline and tag each version.
6. After a baseline, log every change with date, reason and impact.

D6. Common mistakes

- Writing the SRS after the code, so it describes what you hope you built. By week 10 it must match what is deployed.
- Copying or padding the literature review; paraphrase and cite, and compare real platforms.
- Vague requirements (fast, easy, secure, robust) with no number or test. One requirement per statement: if it contains "and" or "or", split it.
- Putting technology choices in requirements; they belong in constraints and design.
- Copy-pasting the same explanation under several diagrams; give each diagram a unique explanation.
- No numbering for requirements, figures and tables; number everything so a reviewer can follow a requirement from need to test.